

Assignment-1: Tools and API Exploration

Here is the comprehensive list of tools, APIs, and libraries used in your `code-agent-v4-ui` project. You can use this information to fill out your assignment table.

1. Google ADK (Agent Development Kit)

Feature	Explanation
Name of tool / API	Google ADK (google-adk)
Link	<i>Internal/Course Resource</i> (Verify source with instructor)
Tool category	Agent Framework / SDK
Purpose	Serves as the backbone for building the autonomous agent, providing the Agent class and model abstractions (LiteLM) to structure the "Plan-Approve-Execute" workflow.
Open-source / Proprietary	Proprietary (likely) or Education Partner Access
Authors / Organization	Google / Google DeepMind
Underlying language	Python
Typical system role	Core Orchestrator. It manages the agent's lifecycle, prompt instructions, and tool execution.
API or interaction style	Python SDK (Importable Module)
Output characteristics	Structured Agent Objects, Tool Calls, and textual responses.
Ease of use	Moderate. Requires understanding of agentic workflows and tool definitions.
Limitations	May depend on specific model compatibilities or internal Google ecosystems.
Relevance to final project	The fundamental framework defining the agent's logic and behavior.

2. Streamlit

Feature	Explanation
Name of tool / API	Streamlit
Link	streamlit.io
Tool category	Web Application Framework (UI)
Purpose	specific interface for interacting with the Code Agent, providing chat windows, sidebars for project management, and settings controls without writing complex frontend code.
Open-source / Proprietary	Open Source (Apache 2.0)
Authors / Organization	Snowflake Inc. (formerly Streamlit Inc.)
Underlying language	Python
Typical system role	Frontend / User Interface. It connects the user's input to the backend Python logic.
API or interaction style	Python Library (Declarative UI)
Output characteristics	Interactive Web Pages (HTML/JS/CSS generated dynamically).
Ease of use	Very High. Allows turning Python scripts into web apps in minutes.
Limitations	Not suited for highly custom complex UI layouts (like full drag-and-drop IDEs) compared to React.
Relevance to final project	The primary user interface for the assignment submission.

3. LiteLLM

Feature	Explanation
Name of tool / API	LiteLLM
Link	litellm.ai
Tool category	Model Abstraction / Proxy

Purpose	Unifies input/output formats across different LLM providers (Gemini, OpenAI, Groq, Ollama), allowing the agent to switch models easily without rewriting code.
Open-source / Proprietary	Open Source (MIT)
Authors / Organization	BerriAI
Underlying language	Python
Typical system role	Middleware. It simplifies API calls to standard OpenAI format.
API or interaction style	Python SDK / Proxy Server
Output characteristics	Standardized JSON responses (matching OpenAI Chat Completion schema).
Ease of use	High. "Drop-in" replacement for OpenAI library.
Limitations	Adds a slight dependency layer; debugging specific provider errors can sometimes be masked.
Relevance to final project	Critical for multi-model support (switching between Local, Google, and OpenRouter models).

4. Google Gemini API

Feature	Explanation
Name of tool / API	Google Gemini API (specifically <code>gemini-2.0-flash-exp</code>)
Link	ai.google.dev
Tool category	Foundation Model API (FM API)
Purpose	Provides the intelligence for the agent to understand code, plan tasks, and generate content. Used as the "Brain" of the agent.
Open-source / Proprietary	Proprietary
Authors / Organization	Google DeepMind
Underlying language	Python (SDK), native C++/TPU execution

Typical system role	Inference Engine. Receives text/prompts and returns generated text/code.
API or interaction style	REST API / Python SDK
Output characteristics	Text, Code, Function Calls (Tool Usage).
Ease of use	High (via google-generativeai or litellm).
Limitations	Rate limits on free tiers; requires API Key.
Relevance to final project	One of the primary high-performance models used for complex reasoning tasks.

5. Tavily Search API

Feature	Explanation
Name of tool / API	Tavily API
Link	tavily.com
Tool category	Search Engine for AI Agents
Purpose	Allows the agent to perform real-time web searches to fetch up-to-date documentation, libraries, or debug errors.
Open-source / Proprietary	Proprietary
Authors / Organization	Tavily
Underlying language	Python (SDK)
Typical system role	External Knowledge Retrieval Tool.
API or interaction style	REST API
Output characteristics	JSON containing summaries and URLs of search results.
Ease of use	High. Designed specifically for LLMs.
Limitations	Monthly request limits on free tier.

Relevance to final project	Enriches the agent's capabilities beyond its training data.
----------------------------	---

6. Ollama

Feature	Explanation
Name of tool / API	Ollama
Link	ollama.com
Tool category	Local Inference Runtime
Purpose	Runs open-source models (like qwen3-coder) locally on your machine, enabling privacy and offline capability.
Open-source / Proprietary	Open Source (MIT)
Authors / Organization	Ollama
Underlying language	Go
Typical system role	Local Model Host.
API or interaction style	CLI / REST API (localhost:11434)
Output characteristics	Streaming Text / JSON.
Ease of use	Very High (simple CLI commands to pull/run models).
Limitations	Performance depends heavily on local hardware (GPU/RAM).
Relevance to final project	Enables the "Local Mode" feature of the Code Agent.

7. Ngrok

Feature	Explanation
Name of tool / API	Ngrok
Link	ngrok.com
Tool category	Tunneling / Networking Tool

Purpose	Exposes the local Ollama API (running on localhost) to the internet so it can be accessed by external services or the agent if running in a cloud notebook.
Open-source / Proprietary	Proprietary (Client is Open Source)
Authors / Organization	Ngrok Inc.
Underlying language	Go
Typical system role	Connectivity Bridge.
API or interaction style	CLI / URL Access
Output characteristics	Public URL forwarding to localhost.
Ease of use	High.
Limitations	Dynamic URLs change on restart (free tier).
Relevance to final project	Essential for making local models accessible to the Streamlit app if deployed remotely or needing SSL.

8. PyGithub & GitPython

Feature	Explanation
Name of tool / API	PyGithub / GitPython
Link	github.com/PyGithub/PyGithub
Tool category	API Wrapper / Utility Library
Purpose	Allows the agent to interact with Git repositories—creating repos, committing code, and pushing changes automatically.
Open-source / Proprietary	Open Source (LGPL / BSD)
Authors / Organization	Independent Contributors (PyGithub), Sebastian Thiel (GitPython)
Underlying language	Python
Typical system role	Version Control integration.

API or interaction style	Python Library
Output characteristics	Git objects (Commits, Trees) or Status codes.
Ease of use	Moderate.
Limitations	Requires careful handling of authentication (Tokens/SSH keys).
Relevance to final project	Powers the automated version control features of the agent.

9. Next.js & Tailwind CSS (Frontend)

(Note: These are found in the *frontend/* directory, distinct from the main *Streamlit* app) | Feature | Explanation | | :--- | :--- | | **Name of tool / API** | Next.js / Tailwind CSS | | **Link** | nextjs.org | | **Tool category** | Web Framework (Frontend) | | **Purpose** | Provide a modern, high-performance user interface (React-based) potentially as a future upgrade to the Streamlit UI. | | **Open-source / Proprietary** | Open Source (MIT) | | **Authors / Organization** | Vercel (Next.js) | | **Underlying language** | TypeScript / JavaScript | | **Typical system role** | Client-side Application. | | **API or interaction style** | React Components / REST API consumption | | **Output characteristics** | Single Page Application (SPA) / SSR HTML. | | **Ease of use** | Moderate (Steep learning curve compared to Streamlit). | | **Limitations** | Requires build steps and separate backend API. | | **Relevance to final project** | Represents the "Advanced UI" component of the project structure. |